



GitHub Copilot Context

How It Thinks

Understanding what Copilot sees — and what it doesn't

The Context Principle

"Context is everything the model can see when generating a response. The model has no memory of previous sessions and no access to files it hasn't been given. Everything it knows comes from the context assembled for the current request."

Inline Completions

Copilot builds a focused context from your cursor area, open tabs, file paths, comments, and repo structure. It selects what's most relevant — it does NOT load the full codebase.

Chat & Agent Mode

Copilot assembles a much richer context: conversation history, semantic search across your repo, explicit references, tool outputs, and web content. It can reach far beyond open files.



Context engineering > Prompt engineering. What Copilot can see matters more than how you phrase the question.

Inline Completions: 6 Context Sources

These are the specific sources Copilot uses when generating Tab-triggered inline suggestions.

1

Cursor Position

Code above and below your cursor — the primary input Copilot uses to predict your next line

2

Comments & Docs

JSDoc, docstrings, inline comments — these are high-signal prompts that strongly guide suggestions

3

Open Tabs

Types, functions, and patterns from files currently open in your editor tabs

4

File Path & Language

Extension, name, directory infer context (e.g., `user.test.js` → testing context)

5

Fill-in-the-Middle (FIM)

Considers both preceding AND following code — not just what's above your cursor

6

Repo Structure

Directory tree and file names — but NOT full file contents unless they are open

Chat & Agent Mode: Context Assembly

Chat assembles a multi-layered prompt. Each layer adds more information for the model to reason about.

1

Your Message

The current prompt you type into the chat input

2

Conversation History

All prior messages in the current session — auto-summarized when the window fills up

3

Implicit Context

Active file, current selection, visible errors, git state — included automatically

4

Explicit References

#file, #codebase, #terminalSelection, @workspace, drag-and-drop, #fetch URLs

5

Semantic Search

Repository indexing finds relevant code by meaning — even files you haven't opened

6

Tool Outputs

In Agent mode: results from file reads, terminal commands, lint errors, test runs

Agent mode adds tools autonomously — it reads files, runs terminal commands, checks errors, and iterates in a loop.

Explicit Context: #-mentions & @-mentions

Type # or @ in the chat input to precisely control what Copilot sees. This is the most powerful lever you have.

Reference	What It Does	Example Usage
#file	Reference a specific file by name	<i>Explain the auth logic in #file:auth.ts</i>
#codebase	Semantic search across entire codebase	<i>How does our app handle errors? #codebase</i>
#terminalSelection	Include terminal output as context	<i>Fix this error #terminalSelection</i>
#selection	Include currently highlighted code	<i>Refactor #selection to use async/await</i>
@workspace	Scope to the full workspace	<i>@workspace find all API endpoints</i>
#fetch	Pull live content from a web URL	<i>Update to .NET 9 #fetch https://learn.microsoft...</i>
Drag & Drop	Drag files/folders into the chat window	<i>Drop a spec file and ask Copilot to implement it</i>
Vision	Attach images (screenshots, UI mockups)	<i>Build a component matching this mockup [image]</i>

Repository Indexing & Semantic Search

Copilot searches your codebase by meaning, not just keywords. This is what makes Chat context-aware.



Auto-Index

When you open Copilot Chat, your repo is automatically indexed in the background. Initial indexing takes up to 60 seconds. Re-indexing is near-instant.



Vector Embedding

Code is converted to vector embeddings that capture meaning. Two functions implementing the same concept are matched, even with different variable names.



Smart Retrieval

When you prompt, Copilot finds the most semantically relevant code and adds it to context — no need to manually specify files.

Key Facts: Built from default branch (main/master) • Shared across all team members • No limit on repos • Works in GitHub.com, VS Code & JetBrains • Agent mode uses it automatically

Context Window: What Affects It

The context window is finite. Even models with 128K–200K+ token windows don't load your entire repo — Copilot strategically selects what's most relevant.



When the context window fills, VS Code auto-summarizes older conversation history. You can also type `/compact` to manually trigger this at any time.

File Length

Long files consume the window quickly — Copilot may not see code far from cursor. Keep files modular (300–500 lines).

Comment Quality

Strategic comments guide suggestions powerfully. Focus on intent ("why"), not just description ("what").

Chat History Length

Long conversations lose early details when summarized. Start fresh threads for new tasks.

Open Files Count

Too many open tabs dilute context with irrelevant code. Close files not related to your current task.

Code Complexity

Deeply nested or verbose code consumes more tokens than clean, well-structured patterns.

Model Choice

Different models have different window sizes and reasoning capabilities. Choose based on task complexity.

What Copilot Cannot See

Critical blind spots you need to account for — these are outside Copilot's reach regardless of mode.



Secrets & Credentials

API keys, passwords — blocked by safety filters



Network / Runtime State

Live API responses, runtime variables, server state



Build Artifacts & Binaries

Compiled output, images, PDFs — not indexed



Private Packages

Internal registries, proprietary library internals



Config Outside Project

System-level configs, global settings files



Full Git History

Recent changes yes, but not full commit log



Environment Variables

Unless referenced in code or comments



Database Schemas

Unless represented in code (e.g., ORM models)

Workaround: Provide these explicitly via comments, Copilot Spaces, or instruction files (covered in later sessions).

Optimize: Inline Completions

Actionable strategies to get better Tab-triggered suggestions

1

Keep Files Modular

Break large files into 300–500 lines so Copilot can see the full file in context.

2

Close Irrelevant Tabs

Only keep files open that relate to your current task. Be intentional about your workspace.

3

Use Clear Function Names

Descriptive names like `calculateCompoundInterest` reduce the need for comments.

4

Write Targeted Comments

Place comments right above where you need suggestions — describe intent, not just behavior.

5

Position Cursor Strategically

Work near relevant code so it's included in context. Copilot reads around your cursor.

6

Use Chat for Complex Tasks

When inline context isn't enough, switch to Chat — it assembles far richer context.

Optimize: Chat & Agent Mode

Maximize Chat and Agent effectiveness with these context strategies

1

Use #-Mentions Explicitly

Reference files with `#file:auth.ts` instead of describing them. Copilot reads what you point to.

2

Start Fresh Threads

New task? New thread. Stale conversation history confuses Copilot. Hit + to reset.

3

/compact for Long Sessions

Type `/compact` to summarize history when chat gets long. Add focus guidance: `/compact focus on API design`.

4

Be Specific, Not Vague

"What does this do" is ambiguous. Say: "Explain error handling in `#file:api.ts` lines 40-60."

5

#fetch for Latest Docs

Reference live documentation with `#fetch URL` to avoid outdated patterns from training data.

6

Let Agent Mode Explore

In Agent mode, give the goal, not the steps. Copilot autonomously reads files, runs commands, iterates.

Preview: Copilot Spaces

A curated context container that makes Copilot act like a subject-matter expert for your team.

What is a Space?

- A secure, shareable container of knowledge + behavioral instructions
- Bundle code files, docs, transcripts, sample queries — anything relevant
- Copilot Chat responses are grounded in the curated Space context
- Available on all Copilot plans (Free, Individual, Business, Enterprise)

Example Spaces

Auth-Model

Authentication patterns, security docs, OAuth config

Data-Queries

SQL examples, schema docs, query optimization notes

Accessibility

WCAG guidelines, component checklists, audit logs

Onboarding

Architecture overview, coding conventions, setup guides

Copilot Spaces: What & Why

A secure, shareable container of knowledge that makes Copilot act like a subject-matter expert.

What is a Space?

- A curated container of code, docs, transcripts, notes, images, and instructions
- Copilot Chat responses are grounded in the Space's curated context
- Sources auto-update — GitHub files always reflect the latest version on main branch
- Available on ALL Copilot plans including Free tier

Why Use Spaces?

Better Answers

Copilot responds with project-specific knowledge instead of generic patterns

Stay in Flow

Collect everything you need for a task in one place — no tab-switching

Reduce Repetition

Share knowledge with your team — avoid explaining the same thing repeatedly

Onboarding

Self-service context that lives beyond chat history for new team members

Creating a Space: Step by Step

1

Navigate

Go to github.com/copilot/spaces and click Create space

2

Name & Owner

Give it a name and choose owner: personal account or organization

3

Add Description

Optional — helps others understand the Space's purpose (not used by Copilot)

4

Write Instructions

Free text telling Copilot what to focus on, its expertise, what to avoid

5

Add Sources

Attach repos, files, PRs, issues, uploads, or free-text content (see next slide)

6

Start Chatting

Ask questions in the Space chat — responses are grounded in your context

Example Instruction:

"You are a SQL generator. Your job is to take the sample queries and data schemas defined in the attached files and generate SQL queries based on the user's goals."

Click Add sources in your Space, then choose from these options:

1

Files & Repositories

Add files, folders, or entire repos from GitHub. When you add a repo, Copilot searches it — but adding specific files gives best results.

2

Links (PRs, Issues, Files)

Paste URLs of GitHub content: pull requests, issues, or specific file links. Great for tracking decisions and discussions.

3

Upload Files

Upload directly from your machine: images, text files, rich documents, spreadsheets. Useful for specs and design docs.

4

Free Text Content

Type or paste content directly: meeting transcripts, notes, architecture decisions, or any relevant context.



Repository vs File — Know the Difference

Attach a repository: Copilot searches and retrieves only the most relevant content per query.

Attach individual files: Full contents loaded into context for EVERY query. Best for critical docs.

Using Spaces: GitHub & IDE

On GitHub.com

- Chat directly inside the Space interface
- Responses are grounded in the Space context
- Change LLM model via the dropdown
- View all conversations in the Conversations tab
- Star favorite spaces for quick access
- Add files from code view via the menu icon

In Your IDE (via MCP)

- Set up the GitHub MCP server in VS Code/JetBrains
- Select Agent from the agent dropdown
- Confirm `get_copilot_space` tool is enabled
- Reference the space in your prompt by name
- Follow-up prompts auto-use the same space
- **Note: repo context not supported in IDE mode**

IDE Example Prompt:

"Using the Copilot space 'Checkout Flow Redesign' owned by myorganization, summarize the implementation plan."

Sharing Spaces

Org-owned: Share with members as Admin, Editor, or Viewer. **Personal:** Share publicly, with specific users, or keep private.

Spaces: Practical Use Cases

Create smaller, purpose-built spaces rather than one giant repo dump.

Feature Work

Add relevant code, product spec, design mockups, and review notes. Ask Copilot to suggest implementation or flag gaps.

Instruction:

You help implement the new checkout flow using React and Tailwind. Reference the spec and existing components.

Repetitive Tasks

Document logic once and share. Copilot suggests patterns, writes templates, reviews alignment with standards.

Instruction:

You help developers implement telemetry events. Validate goals, propose structure from examples, update config.

Knowledge Base

Create spaces for complex systems (auth, CI pipelines). Team members query Copilot for explanations anytime.

Instruction:

You are an expert on our authentication system. Explain code, answer questions, reference latest docs.

Onboarding

Bundle architecture docs, coding conventions, setup guides. New developers get self-service context from day one.

Instruction:

You help new developers understand our codebase. Guide them through setup, architecture, and conventions.

Key Takeaways



Inline completions use focused context: cursor area, open tabs, file paths, comments, and repo structure.



Chat & Agent mode assemble richer context: conversation history, semantic search, explicit references, tool outputs, and web content.



Semantic search finds code by meaning — but Copilot still selects strategically, it never loads the full repo at once.



Context engineering > Prompt engineering. What Copilot can see matters more than how you phrase the question.



Be intentional: use #-mentions, close irrelevant tabs, start fresh threads, write strategic comments, and use /compact.



Coming next: Custom instruction files and Agents will let you provide persistent, team-wide context automatically.